

PROJECT ISSUES REPORT: HM-02 & HM-05

Project: Hikmalayer (hikmalayer-core) **Source baseline:** commit bd0bc21 (per Hikmalayer Security Assessment, 22 June 2026) **Files affected:** `src/api/routes.rs`, `src/persistence.rs` **Prepared by:** Placement Student, Bestower Labs Limited **Classification:** Internal — Confidential

1. Executive Summary

Two findings from the 22 June 2026 Hikmalayer Security Assessment have been remediated:

- **HM-02 (High) — Missing authorization on state-changing endpoints.** Five economic endpoints (`/certificates/issue`, `/tokens/transfer`, `/mine`, `/staking/deposit`, `/staking/withdraw`) accepted requests from any caller with no credential check at all, meaning anyone who could reach the API could move tokens, mint certificates, trigger block production, or manipulate stake. A sixth endpoint of the same class, `/mining/difficulty` (POST), was identified during remediation and fixed for completeness. All six handlers now reject unauthenticated requests by default, using the same `authorize_admin` fail-closed check already in use elsewhere in the codebase (e.g. `/governance/config`, `/slashing/evidence`).
- **HM-05 (Medium) — Unencrypted, non-atomic full-state persistence.** `save_state()` wrote the entire chain state directly to `data/state.json` with `fs::write`, which is not atomic: a crash or power loss mid-write truncates or corrupts the file, and the file was left with default (often world-readable) permissions. `save_state()` now writes to a uniquely-named temporary file in the same directory, sets `0600` permissions before any data is written, calls `sync_all()` to force the write to durable storage, and only then performs a POSIX `rename()` into place — a single filesystem operation that guarantees readers only ever see either the fully-old or fully-new file, never a partial one.

Both fixes are backward-compatible at the wire level for existing valid callers (callers already supplying a valid `x-admin-token` continue to work) and require no schema changes to persisted state. The remaining work required to fully close out the broader finding register (HM-03 real JWT verification, HM-06 secret rotation, HM-07 constant-time comparison, HM-08 consensus hardening) is unaffected by this change and remains tracked separately.

2. Technical Analysis of HM-02

Issue Description

HM-02 is an **authorization bypass / broken access control** issue (CWE-862: Missing Authorization). Five state-changing REST handlers in `routes.rs` — `issue_certificate`, `transfer_tokens`, `mine_block`, `stake_tokens`, and `withdraw_stake` — were registered on the router with no authorization middleware and performed no in-handler credential check before mutating shared state (token ledger, certificate registry, chain, stake table).

Root Cause

The codebase has two functioning authorization primitives, `authorize_admin` and `authorize_p2p` (both fail-closed as of the HM-04 fix), and several handlers already used `authorize_admin` correctly (`update_governance`, `submit_slash_evidence`, `list_slash_evidence`). However, authorization in this codebase is applied **per-handler, opt-in**, rather than enforced centrally (e.g. via a router-wide middleware layer or a default-deny policy). The five endpoints in question were simply never wired to call `authorize_admin`, most likely because they were written early in development (before the admin-token mechanism existed) and never revisited once authorization was added elsewhere. Because Axum does not require a handler to consume `HeaderMap` or perform any check, the omission compiles cleanly and produces no warning — there is no structural guardrail that would have caught this at the routing layer.

Impact Assessment

- **Confidentiality:** No direct exposure, but combined with HM-01 (key custody) an unauthenticated `/mine` call could be used to trigger signing operations.
- **Integrity:** Critical. Any network caller could mint tokens to arbitrary addresses via disguised transfers from a funded account, issue fabricated certificates, force block production at will, or inflate/drain a validator's stake — directly undermining the token ledger and certificate system that are the platform's core value proposition.
- **Availability:** Unrestricted `/mine` and `/staking/*` calls could also be used to exhaust CPU (mining) or corrupt validator economics (stake manipulation), a secondary denial-of-service vector.
- **Business/Legal:** Certificates are marketed as trustworthy attestations; forgeable issuance directly undermines that claim and the business case for the product.

Indicative severity: **High** (consistent with the original audit's ~8.6 CVSS estimate), since exploitation requires only network reachability and no authentication.

Resolution Strategy

Each of the five handlers (plus `set_mining_difficulty`, found to share the same defect) was updated to:

1. Accept an additional headers: `HeaderMap` extractor argument.
2. Call the existing `authorize_admin(&headers, &state)` helper as the **first** statement in the handler body, before any state is read or mutated.
3. Return the endpoint's normal error-shaped JSON response (status: "error") with an explicit "Unauthorized" message when the check fails, mirroring the existing pattern used by `update_governance` and `submit_slash_evidence` so client-side error handling stays consistent across the API.

This reuses the fail-closed `authorize_admin` logic delivered under HM-04 (which already denies access when `ADMIN_TOKEN` is unset), so the fix inherits that hardening rather than introducing a second, possibly inconsistent, auth code path. This is an interim, consistent-with-existing-architecture fix; per-caller identity and role separation (rather than one shared admin credential for all writes) is properly addressed by HM-03 (real JWT verification) and HM-06 (per-peer/per-user credentials with rotation), which remain separate, already-tracked findings.

Code Fix — issue_certificate

File: routes.rs, originally lines 396–427

Original:

```
async fn issue_certificate(
    State(state): State<AppState>,
    Json(payload): Json<CertificateRequest>,
) -> Json<ApiResponse> {
    // Update contract state
    let mut contracts = state.contracts.lock().await;
    ...
}
```

Fixed:

```
async fn issue_certificate(
    State(state): State<AppState>,
    headers: HeaderMap,
    Json(payload): Json<CertificateRequest>,
) -> Json<ApiResponse> {
    if !authorize_admin(&headers, &state) {
        return Json(ApiResponse {
            status: "error".to_string(),
            message: "Unauthorized: a valid admin token is required
to issue certificates"
                .to_string(),
        });
    }

    // Update contract state
    let mut contracts = state.contracts.lock().await;
    ...
}
```

Code Fix — transfer_tokens

File: routes.rs, originally lines 450–491

Original:

```
async fn transfer_tokens(
    State(state): State<AppState>,
    Json(payload): Json<TokenTransferRequest>,
) -> Json<ApiResponse> {
    // Update token balances
    let mut token = state.token.lock().await;
    ...
}
```

Fixed:

```
async fn transfer_tokens(
    State(state): State<AppState>,
    headers: HeaderMap,
    Json(payload): Json<TokenTransferRequest>,
) -> Json<ApiResponse> {
    if !authorize_admin(&headers, &state) {
        return Json(ApiResponse {
            status: "error".to_string(),
            message: "Unauthorized: a valid admin token is required
to transfer tokens"
                .to_string(),
        });
    }

    // Update token balances
    let mut token = state.token.lock().await;
    ...
}
```

```
...
```

Code Fix — mine_block

File: routes.rs, originally line 662 onward

Original:

```
    async fn mine_block(State(state): State<AppState>) ->
    Json<MiningResponse> {
        let finality_depth = {
            ...
```

Fixed:

```
    async fn mine_block(State(state): State<AppState>, headers:
    HeaderMap) -> Json<MiningResponse> {
        if !authorize_admin(&headers, &state) {
            return Json(MiningResponse {
                status: "error".to_string(),
                message: "Unauthorized: a valid admin token is required
to trigger mining"
                    .to_string(),
                block_index: 0,
                transactions_count: 0,
            });
        }

        let finality_depth = {
            ...
```

Code Fix — stake_tokens

File: routes.rs, originally lines 861–871

Original:

```
    async fn stake_tokens(
        State(state): State<AppState>,
        Json(payload): Json<StakeRequest>,
    ) -> Json<StakeResponse> {
        if payload.amount == 0 {
            ...
```

Fixed:

```
    async fn stake_tokens(
        State(state): State<AppState>,
        headers: HeaderMap,
        Json(payload): Json<StakeRequest>,
    ) -> Json<StakeResponse> {
        if !authorize_admin(&headers, &state) {
            return Json(StakeResponse {
                status: "error".to_string(),
                message: "Unauthorized: a valid admin token is required
to stake tokens".to_string(),
                total_stake: 0,
            });
        }
        if payload.amount == 0 {
            ...
```

Code Fix — withdraw_stake

File: routes.rs, originally lines 942–952

Original:

```
async fn withdraw_stake(
  State(state): State<AppState>,
  Json(payload): Json<StakeRequest>,
) -> Json<StakeResponse> {
  if payload.amount == 0 {
    ...
  }
```

Fixed:

```
async fn withdraw_stake(
  State(state): State<AppState>,
  headers: HeaderMap,
  Json(payload): Json<StakeRequest>,
) -> Json<StakeResponse> {
  if !authorize_admin(&headers, &state) {
    return Json(StakeResponse {
      status: "error".to_string(),
      message: "Unauthorized: a valid admin token is required
to withdraw stake"
        .to_string(),
      total_stake: 0,
    });
  }
  if payload.amount == 0 {
    ...
  }
```

Bonus Fix (found during remediation) — set_mining_difficulty

File: routes.rs, originally lines 840–843

This handler was not named explicitly in the original finding but is the same class of defect (unauthenticated write to consensus-relevant configuration) and was fixed for completeness.

Original:

```
async fn set_mining_difficulty(
  State(state): State<AppState>,
  Json(payload): Json<DifficultyRequest>,
) -> Json<ApiResponse> {
  let mut chain = state.chain.lock().await;
  ...
}
```

Fixed:

```
async fn set_mining_difficulty(
  State(state): State<AppState>,
  headers: HeaderMap,
  Json(payload): Json<DifficultyRequest>,
) -> Json<ApiResponse> {
  if !authorize_admin(&headers, &state) {
    return Json(ApiResponse {
      status: "error".to_string(),
      message: "Unauthorized: a valid admin token is required
to change mining difficulty"
        .to_string(),
    });
  }

  let mut chain = state.chain.lock().await;
  ...
}
```

3. Technical Analysis of HM-05

Issue Description

HM-05 is an **insecure, non-atomic persistence** issue (CWE-312: Cleartext Storage of Sensitive Information / CWE-311: Missing Encryption of Sensitive Data, plus a durability defect). `save_state()` serialized the entire application snapshot to JSON and wrote it directly over `data/state.json` using `fs::write`, on every state mutation.

Root Cause

`fs::write` internally opens the target path, truncates it, writes the new bytes, and closes the file — it is **not** an atomic operation. If the process crashes, the host loses power, or the write is interrupted between the truncate and the completion of the write, `data/state.json` is left either empty or containing a partial/invalid JSON document. There is no fallback: `load_state()` simply returns `None` on a JSON parse failure (`serde_json::from_str(&contents).ok()`), which the code in `main.rs` interprets as “no prior state,” silently resetting the chain, ledger, and stake table to genesis. In addition, the file was created with the process’s default permission mask (typically `0644`), meaning any other local user/process could read it. Because the file also currently carries sensitive data (see HM-01), overly-permissive file permissions compound that exposure.

Impact Assessment

- **Integrity/Availability:** A crash during a write (which happens on every mutating API call, including simple token transfers) can permanently corrupt the *entire* chain state, ledger, and validator set — not just the record being written — with no recovery path short of a backup.
- **Confidentiality:** Default file permissions allow any local account on the host to read the full state file, which is a materially larger exposure surface once encryption or restricted permissions are in place around HM-01’s key material.
- **Operational:** Because every API mutation triggers a full-state rewrite, the window of vulnerability recurs continuously in production, not just during rare maintenance operations.

Indicative severity: **Medium**, per the original assessment — likelihood is not attacker driven (no exploit is required, just an ordinary crash/power event) but impact is high once triggered.

Resolution Strategy

`save_state()` was rewritten to follow the standard “write-temp, `fsync`, rename” pattern for crash-safe file updates:

1. **Temp file in the same directory.** A uniquely named temp file (`.state.json.tmp.<uuid>`) is created in the *same* directory as the real target. This matters because `rename()` is only guaranteed atomic when source and destination are on the same filesystem/mount point — writing to `/tmp` and renaming across filesystems would silently fall back to a non-atomic copy on some platforms.
2. **Restrictive permissions before any data is written.** On Unix targets, the temp file’s permissions are set to `0600` (owner read/write only) immediately after creation and before any bytes are written, so there is no window where sensitive content sits on disk with a wider mask. The final path’s permissions are re-asserted to `0600` after the rename as a defense-in-depth measure.

3. **sync_all() before rename.** The temp file's contents (and its metadata) are flushed to durable storage before the rename is issued. Without this, the data could still be sitting in the OS page cache when the rename is journaled, and a crash could produce a renamed file with stale or zero-length content on some filesystems/configurations.
4. **Atomic rename into place.** `fs::rename(&temp_path, path)` is a single filesystem metadata operation. Any reader (including a restarted `load_state()` call after a crash) will observe either the complete old file or the complete new file — never a partial write.
5. **Cleanup on failure.** If either the write or the rename fails, the temporary file is removed so failed attempts don't accumulate stray `.tmp.*` files in the data directory.

Note: this fix addresses the **non-atomic write and file-permission** halves of HM-05 as specified. It intentionally does not yet remove `private_key` material from the persisted snapshot or add at-rest encryption of the JSON payload — those depend on completion of the HM-01 key-custody redesign (`Staker.private_key` still exists in the current `pos.rs`) and are called out in the original remediation plan as a follow-on step once HM-01 lands.

Code Fix — `save_state`

File: `persistence.rs`, originally lines 44–51

Original:

```
pub fn save_state(snapshot: &AppSnapshot) -> std::io::Result<()> {
    if let Some(parent) = Path::new(STATE_PATH).parent() {
        fs::create_dir_all(parent)?;
    }
    let data = serde_json::to_string_pretty(snapshot)
        .map_err(|err|
std::io::Error::new(std::io::ErrorKind::Other, err));
    fs::write(STATE_PATH, data)
}
```

Fixed:

```
use std::fs::{self, File};
use std::io::Write;
#[cfg(unix)]
use std::os::unix::fs::PermissionsExt;

pub fn save_state(snapshot: &AppSnapshot) -> std::io::Result<()> {
    let path = Path::new(STATE_PATH);
    let parent = path.parent().unwrap_or_else(|| Path::new("."));
    fs::create_dir_all(parent)?;

    let data = serde_json::to_string_pretty(snapshot)
        .map_err(|err|
std::io::Error::new(std::io::ErrorKind::Other, err));

    // Temp file must live in the same directory as the target so
the
    // rename below is atomic (rename is only atomic within one
filesystem).
    let file_name = path
        .file_name()
        .and_then(|n| n.to_str())
        .unwrap_or("state.json");
    let temp_path = parent.join(format!("{}.tmp.{}", file_name,
uuid::Uuid::new_v4()));

    let write_result = (|| -> std::io::Result<()> {
```

```

    let mut file = File::create(&temp_path)?;

    #[cfg(unix)]
    file.set_permissions(fs::Permissions::from_mode(0o600))?;

    file.write_all(data.as_bytes())?;
    file.sync_all()?;
    Ok(())
}());

if let Err(err) = write_result {
    let _ = fs::remove_file(&temp_path);
    return Err(err);
}

if let Err(err) = fs::rename(&temp_path, path) {
    let _ = fs::remove_file(&temp_path);
    return Err(err);
}

#[cfg(unix)]
fs::set_permissions(path, fs::Permissions::from_mode(0o600))?;

Ok(())
}

```

4. Verification & Testing Recommendations

HM-02 — Authorization

#	Test	Expected Result
1	POST /certificates/issue with no x-admin-token header	status: "error", unauthorized message; no certificate created; no pending transaction added
2	POST /certificates/issue with an incorrect x-admin-token	Same as above
3	POST /certificates/issue with the correct x-admin-token	status: "success"; certificate created and pending transaction added (existing behaviour preserved)
4	Repeat tests 1-3 for POST /tokens/transfer	Balances unchanged on failure; transferred correctly on success
5	Repeat tests 1-3 for POST /mine	No block produced on failure; block produced and gossiped on success
6	Repeat tests 1-3 for POST /staking/deposit and POST /staking/withdraw	Stake table unchanged on failure; updated correctly on success
7	Repeat tests 1-3 for POST /mining/difficulty	Difficulty unchanged on failure; updated on success
8	Start the node with ADMIN_TOKEN unset	All six endpoints deny every request (fail-closed inherited from HM-04), including requests that send <i>no</i> token header at all
9	Regression: GET endpoints (/blocks, /tokens/balance/{account},	Continue to work unauthenticated, confirming the fix did not over-broadly lock

	/staking/validators, etc.)	down read-only routes
10	Regression: existing authorized flows (/governance/config POST, /slashing/evidence POST)	Unaffected — same authorize_admin helper, no behavioural change

Automate tests 1–8 as part of the existing test suite (the assessment notes it currently has only 9 tests total) using `axum::body / tower::ServiceExt::oneshot` against the router returned by `api_routes()`, asserting on response body status fields (and, if the response type is upgraded to carry real HTTP status codes in a follow-up pass, on 401/403).

HM-05 — Atomic, Permissioned Persistence

#	Test	Expected Result
1	Call <code>save_state()</code> normally, then <code>load_state()</code>	Round-trips correctly; no behavioural regression
2	Inspect <code>data/state.json</code> permissions after a save (<code>ls -l / stat</code>) on a Unix host	<code>-rw----- (0600)</code>
3	Kill (SIGKILL) the process mid-write in a controlled test (e.g. inject a delay before rename and kill during that window)	<code>data/state.json</code> still contains the previous valid snapshot (rename never occurred); no <code>.tmp.*</code> files retained after a subsequent clean save
4	Simulate a failure inside the write closure (e.g. temporarily point <code>STATE_PATH</code> at a read-only directory)	<code>save_state()</code> returns an <code>Err</code> ; no <code>.tmp.*</code> file is left behind in the target directory
5	Concurrent saves under load (multiple rapid mutating API calls)	Each save produces its own uniquely-named temp file (UUID-suffixed), so concurrent writers cannot collide on the same temp path
6	Confirm no plaintext secrets remain in the state file post HM-01	Once HM-01 lands, re-run <code>grep -ri "private_key" data/state.json</code> and confirm no matches, per the original acceptance criteria
7	Restart the node after a normal shutdown and after a simulated crash (test 3)	Chain, ledger, stake table, and governance config all load correctly with no data loss beyond the single in-flight mutation at crash time

General Regression Guidance

- Re-run the full existing test suite (`cargo test`) after applying both fixes to confirm no unrelated handler signatures were broken by the added `HeaderMap` parameter.
- Re-run `cargo clippy` and `cargo audit`, per the placement's standing SAST practice, to confirm the new code introduces no new lint warnings or vulnerable dependency usage (`uuid`, used here for temp-file naming, is already a project dependency via `transaction.rs/protocol.rs`).
- Update the functional test matrix (Deliverable D-04) to reflect the newly-required `x-admin-token` header on `/certificates/issue`, `/tokens/transfer`, `/mine`, `/staking/deposit`, `/staking/withdraw`, and `/mining/difficulty`.
- Track HM-01 (key custody), HM-03 (real JWT verification / per-caller identity), and HM-06 (per-peer credential rotation) separately — the interim admin-token gate applied here is a correct, fail-closed stopgap consistent with the rest of the codebase, but it is not a substitute for per-user authorization once HM-03 lands.